FunC v0.5.0 → Fift
Feature-by-Feature Lowering Guide

Tomi Setsu

April 19, 2026

Abstract

This document complements `doc/func_v0.5.0.tex` by explaining how the proposed FunC `0.5.0` surface can be lowered into Fift/TVM code without introducing a hidden runtime or a second virtual machine. The goal is to make the lowering model explicit enough that language design can be judged against the real constraints of TVM/TOS.

The central claim is that FunC `0.5.0` should remain a *front-end heavy, runtime-light* language. Most new features are either:

1. compile-time only and erased before code generation;
2. mapped to fixed TVM stack or cell representations;
3. implemented by generated wrappers around existing TVM conventions.

This is a design-lowering specification, not a statement that the current compiler already implements all of the described transformations.

Introduction

The proposed FunC `0.5.0` surface is intentionally more expressive than FunC `0.4.6`: it adds named structs, enums, message declarations, `Option`/`Result`, explicit mutability, explicit effect sets, traits, generics, typed contexts, and typed cell APIs. The obvious concern is whether these features remain faithful to the low-level execution model.

They can, provided that the compiler obeys three discipline rules:

1. no hidden heap or object runtime;
2. no dynamic dispatch requirement;
3. no source feature whose meaning cannot be reduced to ordinary TVM stack, cell, control-flow, and action-phase operations.

This document therefore answers a practical question: *for each major FunC 0.5.0 feature, what is the canonical lowering path to Fift/TVM?*

Contents

1 Scope and Backend Model

1.1 What "Lowering to Fift" Means

In this document, "lowering to Fift" means lowering to the same TVM execution model already targeted by current FunC and Fift macro assembly. The backend may internally pass through an IR, but the observable result should still be a selector-based TVM program expressible in Fift assembly.

In particular:

- ordinary functions still become TVM routines callable through selector indices and the existing `CALLDICT`/`JMPDICT` model;
- cell parsing still becomes `CTOS`, `LDU`/`LDUX`, `LDI`/`LDIX`, `LDREF`, and related instructions;
- cell building still becomes `NEWC`, `STU`/`STUX`, `STI`/`STIX`, `STREF`, and `ENDC`;
- outbound messaging still ends in `SENDRAWMSG`;
- persistent state still goes through the existing data-register conventions and helpers such as `get_data`/`set_data`.

1.2 Three Representation Layers

Every source construct must be understood at three different layers:

1. **source representation**: what the author writes;
2. **runtime representation**: what exists on the TVM stack while code executes;
3. **wire/storage representation**: how values are serialized into cells for messages, persistent state, dictionaries, or ABI boundaries.

Many confusions disappear once these layers are kept distinct. For example, `Option<T>` need not have the same representation on the stack and on the wire, as long as the compiler-generated codecs are deterministic.

1.3 Canonical Lowering vs Optimization

This document specifies a *canonical* lowering. Optimizations are still allowed afterwards:

- tuples may be scalarized into separate locals;
- enum tests may become jump tables or branch trees;
- `Option<T>` may be unboxed locally when the optimization is not observable;
- small helper functions may be inlined;
- dead fields or dead temporary tuples may be removed.

The important invariant is that optimization may improve code generation, but must not change the source-level meaning or the externally visible wire format.

2 Lowering Principles

2.1 Compile-Time Features Must Erase

Several proposed `0.5.0` features exist to improve readability and static safety, not to create a runtime subsystem. In particular:

- `mod`, `use`, and `pub` are namespace features only;
- effect sets are checked statically and erased;
- mutable-place overlap checks are static only;
- traits and generics use static dispatch and monomorphization;
- field names and variant names are debug/source concepts, not runtime symbols.

2.2 Runtime Features Must Map to Existing TVM Values

When a feature survives past type checking, it must lower to existing TVM value kinds: integers, cells, slices, builders, tuples, continuations, or null. Nothing in the proposal requires a new runtime type.

2.3 Protocol-Aware Wrappers Are Allowed

Some source forms correspond directly to blockchain protocol concepts rather than to ordinary library code. These may use generated wrappers:

- entry qualifiers `internal fn`, `external fn`, `bounce fn`, `get fn` inside a `contract { }` block;
- typed message parameters on entries (`internal fn recv(msg: T)`); the stdlib-provided `Signed<T>` and `BounceContext<T>` wrappers;
- `message` declarations with bound opcodes;
- `BodyLayout::Auto` and typed `send(... )`.

This is acceptable because these features do not invent a second semantic world; they merely expose real TVM/TOS conventions in a safer surface form.

2.4 Static Dispatch Is Mandatory

To keep the backend simple and predictable:

- trait calls must resolve at compile time;
- generic functions must be monomorphized;
- there are no trait objects, no vtables, and no hidden dictionary-passing runtime.

3 Canonical Runtime Representations

3.1 Scalars and Domain Types

Source type	Canonical TVM stack representation	Wire/storage representation
`iN`/`uN`	ordinary TVM integer	fixed-width or variable-width integer encoding, depending on codec
`bool`	ordinary TVM integer, normalized to `0` for false and `-1` for true	usually a 1-bit or fixed small-integer encoding chosen by the relevant codec
`Coins`	ordinary TVM integer with source-level domain meaning	`Grams`/`VarUInteger` encoding through the standard amount codecs
`Address`	ordinary TVM `Slice` carrying the invariant "valid serialized address"	serialized message address form
`RawCell`	ordinary TVM `Cell`	ordinary cell reference
`Cell<T>`	ordinary TVM `Cell`; `T` is erased after type checking	same raw cell bits and refs, but interpreted through `T`'s codec
`RawSlice` / `Slice`	ordinary TVM `Slice`	not directly stored; arises from `CTOS` or message/body parsing
`Builder`	ordinary TVM `Builder`	not directly stored; finalized with `ENDC`
`Bytes`	byte-aligned `Slice` or stdlib wrapper over a slice/cell pair	byte-string codec; exact layout must be fixed by the stdlib/ABI

Source type	Canonical TVM stack representation	Wire/storage representation
`()`	zero stack values at function boundaries; null/empty payload when embedded	zero-width payload or omitted payload by codec rule

3.2 Aggregates and Sum Types

Source construct	Canonical TVM stack representation	Wire/storage representation
`(T1, ..., Tn)`	TVM tuple or flattened multi-value temporary	codec-dependent tuple/product layout
`struct S {...}`	logical tuple in field order; backend may scalarize locals	derived product codec in field order
`enum E {...}`	tagged tuple `(tag, payload)`; payload is a tuple or null	tag plus payload, with tag width fixed by the derived codec
`Option<T>` / `T?`	tagged tuple like any two-variant enum; optimizer may unbox locally	`Maybe`-style tag plus payload
`Result<T, E>`	tagged tuple `(0, ok)` or `(1, err)` canonically	tag plus payload if explicitly serialized
`message M(op = ...){...}`	same as a named struct at runtime, plus compile-time opcode metadata	opcode prefix followed by field encoding

4 Feature-by-Feature Lowering Table

Feature	**Frontend handling**	**Canonical Fift/TVM lowering**	**Notes**
`mod`, `use`, `pub`	resolve names, visibility, and imports at compile time	no runtime code; generate mangled function/type symbols only	pure erasure
`const`, type alias	evaluate or resolve during semantic analysis	constant immediates or no code at all	pure erasure except constant uses
`let`	create immutable local binding	ordinary local slot / stack temporary	no runtime mutability bit
`let mut`	mark local as assignable	same local slot shape as `let`	`mut` is checker-only
assignment to locals/fields	verify place mutability and field existence	stack shuffles plus tuple rebuild or scalar local update	backend may scalarize struct fields
`bool`	type-check boolean-only operators and conditions	emit ordinary integer-producing comparisons; normalize to `0`/`-1`	no new VM type
`Coins`	track domain meaning distinctly from raw ints	ordinary TVM integer in computation; amount codec on serialization	zero-cost in arithmetic paths
`Address`	type-check address-only APIs	ordinary slice value plus address-validity invariant	codec uses message-address helpers
`Cell<T>`	enforce typed cell APIs and erase `T` after checking	ordinary cell value; codecs inserted at load/store boundaries	no runtime generic metadata
`struct`	resolve field offsets and patterns	tuple-like product value or flattened locals	field names erased after lowering
`enum`	assign canonical tags and verify exhaustiveness	tagged tuple `(tag, payload)`; branch on tag	payload may be null for zero-field variants

Feature	Frontend handling	Canonical Fift/TVM lowering	Notes
`message(op = X)`	derive codec and bind opcode metadata	generated decoder checks prefix, then parses fields; encoder prepends opcode	protocol-aware wrapper justified
`Option<T>` / `T?`	desugar `T?` to `Option<T>`	two-variant tagged tuple canonically	optimizer may unbox locally
`Result<T, E>`	type-check recoverable error paths	two-variant tagged tuple canonically	drives `?` lowering
`trait` / `impl`	resolve methods and trait obligations statically	no runtime object; selected concrete function symbol only	no vtables
generics	instantiate per concrete type vector	monomorphized concrete functions/types, then ordinary lowering	same discipline as template expansion
`mut self` methods	desugar method call into explicit state threading	internal helper returns updated receiver plus result, then uses existing TVM ops	surface replacement for `~method`
`mut` arguments	verify mutable-place rules	same calling convention as explicit state threading or by-value update	overlap checks are static only
`if let`	desugar to match over one scrutinee	tag/type test plus destructuring branch	for `Option`, often just tag test
`while let`	desugar to loop plus match/test	loop header, test, destructure, exit branch	no special runtime form
`match`	check exhaustiveness, bind arm patterns	discriminant test tree, jump table, or direct destructuring	message opcodes may use preload fast path
`?` on `Result`	check enclosing residual type	branch on tag; on error construct early return and jump to epilogue	zero hidden exceptions

Feature	Frontend handling	Canonical Fift/TVM lowering	Notes
`effect(...)`	check allowed operations transitively	erased before code generation	may survive only as debug metadata/comments
`extern "tvm"`	bind a source signature to a raw intrinsic	emit direct intrinsic/opcode sequence or imported asm helper	surface replacement for parser magic
`internal fn recv(msg: T)`	generate ABI wrapper that peeks 32-bit opcode, decodes body into T	existing internal-entry ABI + opcode match + field decode + user body call	unknown op → silent drop
`external fn recv(msg: Signed<T>)`	generate ABI wrapper that decodes body into T, signature stays in wrapper	existing external-entry ABI plus derived-codec parse	`accept_message` is the author's responsibility
`bounce fn on_bounce(ctx: BounceContext<T>)`	generate wrapper for bounced-message path; strip 32-bit header	existing bounced-message convention plus bounce-body decoding	T subject to 256-bit / 0-ref rule
`get fn`	mark as getter export and read-only surface	ordinary getter selector / method-id export	same getter ABI family as today
`Signed<T>` / `BounceContext<T>`	stdlib wrappers, no compiler magic; derived codec like any message	wrapper fields are unpacked by ordinary field-load sequence	`Signed<T>::check` calls `CHKSIGNU`
`to_cell/from_cell`	call `CellEncode/CellDecode` implementation	`NEWC` + stores + `ENDC`; or `CTOS` + loads + `ENDS`	typed facade over raw cell ops
`load_any/store_any`	resolve concrete codec statically	call monomorphized codec helper, which emits raw load/store instructions	no reflective runtime
`send<T>`	resolve body codec and send options	encode body, build message envelope, choose inline/ref layout, emit `SENDRAWMSG`	typed wrapper over action-cell construction

Feature	Frontend handling	Canonical Fift/TVM lowering	Notes
`BodyLayout::Auto`	track layout policy in source	generated size-aware branch chooses inline body or ref body deterministically	must be fully deterministic
`BounceMode`	select bounce policy at source level	maps to message-header bits and chosen bounced-body convention	no dedicated VM primitive
`for item in items`	desugar through a statically resolved iterator protocol	hidden iterator state plus `loop` and `next()` match	initial implementation should restrict iterable kinds

5 Detailed Lowering Notes

5.1 Functions Still Lower to Selector-Based TVM Routines

After desugaring, ordinary functions should still lower to the same broad model used today:

1. each concrete function body is assigned a selector;
2. the program stores its selector dispatcher in `c3`;
3. intra-program calls become `CALLDICT n` or equivalent selector calls;
4. tail calls become `JMPDICT n` or equivalent jumps.

The proposed language features therefore sit *above* the current function family model; they do not replace it.

5.2 Explicit Mutability Lowers to Explicit State Threading

A method with `mut self` does not require a new calling convention at the VM level. Under the abort-by-default error model, stdlib `mut self` methods return raw values (not `Result`); on failure they `THROW`, which unwinds the whole transaction so no per-place rollback logic is needed.

Canonical desugar for the common case:

```
let mut cs = body;
let op = cs.load_uint(32);    // aborts on cell underflow

// canonical desugar:
let (__cs1, __v) = __Slice_load_uint(cs, 32);
cs = __cs1;                   // commit new receiver
let op = __v;                 // bind value
```

`__Slice_load_uint` lowers directly to the `LDUX`-based helper family already used by current `~load_uint` built-ins. The helper `THROW`s on underflow; no `match` / tag check is emitted in the desugar.

The rare `Result`-returning `mut self` case. When a `mut self` method genuinely returns `Result<T, E>` (uncommon in 0.5.0), the caller binds the result via `match`:

```
let mut cs = body;
match cs.try_load_uint(32) {
    Ok(v)  => { /* cs already committed to __cs1 */ ... }
    Err(e) => { /* cs value is still __cs1 (committed);
                   authors who need a snapshot must copy cs before the call */ ...
}
```

The unconditional writeback is acceptable here because `Result`- returning `mut self` methods are rare and their authors may document the partial-mutation semantics.

5.3 Structs Lower as Products, Not Objects

Structs do not imply heap allocation or object identity. Their canonical meaning is just "a named product type". The lowering strategy is:

1. field names guide parsing, typing, and pattern matching;
2. a struct value is represented as a tuple in field order at observable boundaries;
3. within one function, the backend may flatten fields into separate locals.

This keeps structs zero-cost in the common case while still giving the user a legible source-level model.

5.4 Enums, Option, and Result Lower as Tagged Products

General algebraic data types require an explicit discriminant. The canonical runtime form is therefore:

```
(tag, payload)
```

where:

- `tag` is an integer discriminant;
- `payload` is a tuple of the selected variant fields, or null for a zero-field variant.

Examples:

```
Option<T>::None         => (0, null)
Option<T>::Some(x)      => (1, x)
Result<T, E>::Ok(x)     => (0, x)
Result<T, E>::Err(e)    => (1, e)
```

This representation is uniform, supports nesting, and matches ordinary `match` lowering. A backend may locally optimize some cases to a more compact form, but the canonical semantic model should remain tagged.

5.5 Pattern Matching Lowers to Tag Tests and Destructuring

A `match` expression lowers in four stages:

1. evaluate the scrutinee into a temporary;
2. extract or inspect its discriminant;
3. branch to the selected arm;
4. destructure the payload into arm-local bindings.

For dense integer discriminants, the backend may generate a jump table. For sparse discriminants, it may generate a branch tree. For message decoding, it may first use a slice preload such as `PLDUZ 32` or a full `LDU 32` to inspect the opcode before decoding the remainder of the body.

5.6 `require`, `revert`, and Abort Lowering

The main failure mechanism in FunC `0.5.0` is abort. The two source constructs `require(cond, Err)` and `revert(Err)` both lower to the TVM `THROW` / `THROWIF` / `THROWIFNOT` family.

`require(cond, Err).` Lowers to a conditional throw:

```
require(cond, Err)
  ==>
  <evaluate cond>
  <push (Err as u16)>
  THROWIFNOT
```

Equivalent to `if !(cond) { THROW(Err as u16) }`.

`revert(Err).` Lowers to an unconditional throw:

```
revert(Err)
  ==>
  <push (Err as u16)>
  THROW
```

`revert(Err)` is semantically equivalent to `require(false, Err)`; the compiler folds either form to the same single-instruction sequence.

Where the error identifier resolves to a `u16` constant at compile time (from an `error` shorthand or an `enum E : u16` explicit discriminant), the compiler prefers the `THROWIFNOT n` or `THROW n` immediate-operand form when available.

Stdlib functions that abort on failure. Stdlib APIs such as `Slice::load_uint`, `Builder::store_uint`, `Dict::get`, and `send` return raw values; failure is signalled by TVM-level abort. The lowering is typically the bare TVM opcode with no extra guard:

```
cs.load_uint(32)
  ==>
  <cs>
  LDU 32           // TVM opcode; throws 9 on underflow
```

The compiler emits no `Result` wrapper, no tag check, and no control-flow split. On abort, the transaction wrapper's handler picks up the thrown code and propagates it as the contract's exit code.

`Result<T, E>` (rare use) still lowers as a tagged product. When a function genuinely returns `Result<T, E>`, the tagged product from Canonical Decision 1 appears on the stack as `(tag, payload)`. The caller destructures it with `match`, which lowers to a tag test and a branch — ordinary enum lowering, not a new form.

`emit` lowering. `emit Event \{ fields \}` lowers to:

```
send(LOG_SINK_ADDR, Event { fields }, EVENT_DEFAULT_OPTS)
```

which in turn lowers through §5.12 below.

5.7 Typed Messages Lower to Generated Codecs

A declaration such as

```
pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
}
```

should generate at least:

1. a stack-level runtime type equivalent to a struct;
2. an encoder that prepends `0x0f8a7ea5` and then serializes fields;
3. a decoder that checks the prefix and then parses fields;
4. metadata allowing typed entrypoints and typed `send` wrappers to select the codec automatically.

The encoder lowers to ordinary builder instructions such as `NEWC`, `STU`/`STUX`, address stores, amount stores, optional ref stores, and `ENDC`. The decoder lowers to `CTOS`, `LDU`/`LDUX`, address loads, amount loads, optional-value loads, and a final `ENDS` when the codec requires exact consumption.

5.8 Typed Cell APIs Lower to Raw Cell Operations

The typed cell APIs are wrappers over codecs and raw cell primitives:

- `to_cell(x)` becomes "encode `x` into a builder, then `ENDC`";
- `from_cell(c)` becomes "`CTOS`, decode fields, ensure final slice invariants, return typed value";
- `load_any<T>` becomes a statically resolved call to `T`'s decoder;
- `store_any<T>` becomes a statically resolved call to `T`'s encoder.

The important point is that `Cell<T>` itself carries no runtime metadata. All real work happens at codec boundaries.

5.9 Typed Entry Parameters Lower to ABI Wrappers

A function such as

```
contract Jetton {
    internal fn on_transfer(msg: Transfer) { ... }
}
```

lowers to two layers:

1. a generated low-level entry wrapper with the existing internal-message ABI;
2. a typed user function body that receives a decoded `Transfer`.

The wrapper:

1. reads the raw inbound context values from the existing ABI inputs;
2. binds `ctx.sender`, `ctx.value`, `ctx.forward_fee` to internal wrapper-local names that the user body can reference;
3. peeks the first 32 bits of the body for the opcode and matches against `Transfer`'s declared op;
4. on match, decodes the remaining fields and calls the user body;
5. on mismatch, exits successfully without invoking the user body;
6. on successful return from the user body, commits state via `SETDATA` and terminates with exit code 0;
7. on any `THROW` during the user body, skips the state commit and propagates the exit code.

For `external fn recv(msg: Signed<Payload>)` the wrapper applies the same model plus signature decoding. For `bounce fn on_bounce(ctx: BounceContext<T>)` the wrapper first strips the `0xFFFFFFFF` bounce header.

For a `Slice` entry parameter the wrapper skips decoding entirely; the user body receives the raw body slice.

Trailing `Slice` / `Cell` field handling. A trailing `Slice` field on a message type captures the remaining bits and refs of the cell *unconsumed* by the structured fields. Lowering: after decoding the structured fields, the remainder slice is bound directly to the field name without further `ENDS` check. A trailing `Cell` / `RawCell` field consumes one more `^Cell` reference via `LDREF` instead.

`enum` of messages for dispatch. A wrapper whose parameter is an enum of `message` variants performs a 32-bit opcode peek once, matches against the concatenated variants' opcode set, and dispatches to the matching variant's decoder + user-body arm. No match → silent success.

This is protocol-aware lowering, but it is still only a wrapper around the real VM inputs.

5.10 Effect Sets Do Not Survive Code Generation

An effect annotation such as

```
effect(read, write, send)
```

should be enforced entirely by static analysis:

- a `pure` function may not reach `get_data`, `set_data`, `SENDRAWMSG`, raw asm, or unsafe intrinsics;
- a `read`-only function may read context and storage but not write or emit actions;
- a `send`-annotated function may use outbound-message primitives.

After validation, the annotation can be erased. The generated Fift code does not need a runtime effect table.

5.11 Traits and Generics Lower by Monomorphization

Trait calls must never require a runtime interface object. The lowering model is:

1. resolve the concrete implementation during type checking;

2. create a concrete instantiation of any generic function or method;
3. compile the resulting concrete function like any other function.

For example, a call to:

```
send<Transfer>(dst, body, opts)
```

should resolve the `CellEncode` implementation for `Transfer` at compile time and produce a concrete body encoder call, not a reflective or dynamic dispatch sequence.

5.12 Typed Send Lowers to Ordinary Message Construction

The high-level call

```
send(dst, body, opts)
```

should lower to the following pipeline:

1. encode `body` using the selected codec;
2. build the message envelope with the right destination, amount, and flags;
3. decide whether the body is inlined or stored by reference;
4. finalize the message cell;
5. emit `SENDRAWMSG` with the chosen send mode.

`BounceMode` and `BodyLayout` are therefore not new runtime entities. They are source-level policy inputs to message construction.

5.13 BodyLayout::Auto Must Be Deterministic

`BodyLayout::Auto` is useful only if all compliant toolchains make the same choice for the same source and codec. The canonical rule should therefore be:

1. compute the encoded body size exactly when it is statically known;
2. otherwise encode into a temporary builder/cell first;
3. choose inline vs ref according to a fully specified fit predicate;
4. emit the corresponding branch or specialized fast path.

The exact fit predicate belongs to the language+stdlib contract and must be stable across compilers.

5.14 `asm` Blocks Pass Through to Fift

`asm` blocks are the only surface form whose content bypasses type checking and lowering. The compiler's contract is to:

1. verify the enclosing function carries the `unsafe` qualifier;
2. for inline `asm(in\_vars) -> (out\_tys) { """..." "" }`, arrange the named input variables on the TVM stack per the declared order before emitting the literal Fift snippet;
3. emit the Fift tokens verbatim into the generated Fift source (with appropriate escaping);
4. after the block, treat the stack as the declared `out\_tys` and bind to outer temporaries or return values accordingly;
5. for function-body `asm "..."` or `asm { """..." "" }` forms, the rule in `crypto/func/parse-func.cpp` §5.4 for 0.4.6 `asm` bodies applies: width constraints (≤ 16 per argument and per return tensor) and optional argument/return order permutation are honoured.

The compiler does not parse the Fift beyond minimal tokenisation (balanced quotes, comment handling). The burden of correctness lies with the author.

Effect inference inside `asm`. Because the compiler does not understand the Fift, it cannot infer an effect set for an `asm` block. The enclosing function therefore defaults to the `unsafe` qualifier's semantics: the compiler treats it as if it had the maximal effect set `effect(read, write, send)` for the purpose of checking other callers. Authors who want a safe wrapper provide a non-`unsafe` function that calls the `unsafe` primitive with a narrower declared effect set.

6 Worked Lowering Example

6.1 Source Fragment

```
contract Wallet {
    state { seqno: u32, owner: Address }

    error WrongSeqno = 33,
          ZeroAmount = 34;

    message Transfer(op = 0x0f8a7ea5) {
        seqno:  u32,
        to:     Address,
        amount: Coins,
    }

    internal fn on_transfer(msg: Transfer) {
        require(msg.seqno == state.seqno,   WrongSeqno);
        require(msg.amount > Coins::zero(), ZeroAmount);

        send(
            msg.to,
            (),
            SendOptions {
                value:       msg.amount,
                bounce:      BounceMode::Body256,
                body_layout: BodyLayout::Auto,
                mode:        SendMode::PayFeesSeparately,
            }
```

```
        );

        state.seqno += 1;
    }
}
```

6.2 Conceptual Lowering

The canonical lowering is:

1. generate `__Transfer_encode` and `__Transfer_decode`;
2. generate error enum `WalletError` → `u16` mapping;
3. generate a low-level internal-entry wrapper `__entry_on_transfer`;
4. keep the typed user body as a concrete selector function;
5. lower each `require` to an `IFNOT` + `THROW` sequence with the numeric error code;
6. lower `send(... )` to ordinary message building plus `SENDRAWMSG`;
7. lower `state.seqno += 1` to a tuple-field rewrite on the wrapper-supplied `state` local;
8. emit `SETDATA` on normal return, nothing on `THROW`.

6.3 Generated Wrapper Shape

```
__entry_on_transfer(low_level_abi_args):
    raw_ctx  = parse_low_level_internal_context(low_level_abi_args)
    op       = preload_u32(raw_ctx.body_slice)
    if op != 0x0f8a7ea5:
        return 0                    # silent drop, no state commit
    msg      = __Transfer_decode_after_op(raw_ctx.body_slice)
    # bind wrapper-local ctx.sender, ctx.value, ctx.forward_fee
    state    = load_state()          # aborts on malformed state cell
    # call the user body, which mutates state via the wrapper frame
    __user_on_transfer_impl(state, msg, raw_ctx)
    save_state(state)                # SETDATA on normal return only
    return 0
```

6.4 User Body Shape After Desugaring

```
__user_on_transfer_impl(state, msg, ctx):
    if !(msg.seqno == state.seqno):
        THROW 33                        # WrongSeqno
    if !(msg.amount > 0):
        THROW 34                        # ZeroAmount

    send(msg.to, (), opts_from(msg.amount))  # aborts on any failure

    state.seqno = state.seqno + 1
```

6.5 Fift/TVM Shape

At the final backend stage, the remaining building blocks are ordinary TVM operations:

- state load/save via `GETDATA` and `SETDATA`;
- integer comparison for the seqno check;
- builder construction and `ENDC` for the encoded outbound message body;
- `SENDRAWMSG` for the outbound transfer;
- selector-based function calls for helper routines;
- `THROWIFNOT` for each `require`;
- conditional branches only for `match` over enums and `if let` over `Option`.

In other words, the high-level source is reduced to the same machine model already used by present FunC and hand-written Fift — no hidden runtime, no reflective dispatch, no `Result` control flow.

7 Canonical Decisions Fixed

Every ABI-level and semantic decision that earlier drafts of this document listed as *pending* has now been fixed in the main specification. The authoritative statements are in `doc/func_v0.5.0.tex` §“Canonical Decisions”:

1. stack representation of `enum`, `Option<T>`, `Result<T, E>`: main-spec Canonical Decision 1;
2. wire format of serialized enums, `Option`, and `Result`: main-spec Canonical Decision 2;
3. canonical layout of `Bytes`: main-spec Canonical Decision 3;
4. `BodyLayout::Auto` fit predicate: main-spec Canonical Decision 4;
5. abort semantics and `require`-to-exit-code mapping: main-spec Canonical Decision 5;
6. getter / `method_id` selector policy: main-spec Canonical Decision 6;
7. expression grammar and operator precedence (no `?`): main-spec Canonical Decision 7;
8. trait coherence and orphan rules: main-spec Canonical Decision 8;
9. entry parameter is a typed message (replaces lazy/eager distinction): main-spec Canonical Decision 9;
10. `0.4-compat` edition surface and cross-edition rules: main-spec Canonical Decision 10.

The backend lowering described earlier in this document is consistent with each of those canonical rules; implementations MUST obey the main-spec wording verbatim at ABI boundaries.

8 Conclusion

FunC `0.5.0` can remain fully lowerable to Fift/TVM if it preserves one discipline: *push complexity into the front end, not into the runtime.*

Under that discipline:

- modules, visibility, effects, and mutable-place checks erase entirely;
- structs, enums, messages, and results become fixed tuple/cell layouts;

- typed entrypoints and typed send wrappers become generated adapters over existing protocol conventions;
- traits and generics remain static-dispatch only;
- the final emitted operations are still the familiar TVM primitives for control flow, parsing, building, state access, and `SENDRAWMSG`.

This is exactly the kind of language upgrade that keeps TVM/TOS honest: better source ergonomics, stronger static guarantees, and no semantic break from the underlying machine.